

UCUENCA

UNIVERSIDAD DE CUENCA

DEET — Maestría en Ciencias de la Ingeniería Eléctrica

Comparación de Enfoques K-Means: Inferencia de VLANs vs. Mantenimiento Robótico

2026-06-03

Comparación de Enfoques K-Means: Inferencia de VLANs vs. Mantenimiento Robótico

Universidad de Cuenca | DEET | Maestría en Ciencias de la Ingeniería Eléctrica
Fecha: 2026-06-03

1. Contexto General

Este documento compara dos aplicaciones independientes del algoritmo K-Means implementadas en Julia, ambas desarrolladas en el marco del curso de Redes Complejas. Aunque comparten el mismo núcleo algorítmico, difieren en dominio, features, objetivo y criterios de evaluación.

Dimensión	Proyecto A — Inferencia de VLANs (Jean)	Proyecto B — Mantenimiento Robótico (Henry)
Dominio	Redes de computadoras	Robótica industrial
Objetivo	Inferir segmentación lógica de red sin configuración de switches	Clasificar actuadores por estado de mantenimiento
Datos	Tráfico de red sintético (flujos IP)	Inventario de actuadores con telemetría de sensores
Salida del modelo	Mapa IP → VLAN inferida	Mapa pieza → estado (Programado / Urgente / Reemplazo)
Carpeta	Kmeans_Robot_Jean/	Kmeans_Robot_Henry/

2. Metodología — Proyecto A: Inferencia de VLANs

2.1 Descripción del Problema

En redes corporativas, las VLANs segmentan el tráfico para mejorar seguridad y rendimiento. Cuando no se dispone de la configuración del switch, es posible **inferir la segmentación lógica** analizando los patrones de comunicación entre hosts directamente desde capturas de tráfico (PCAP).

Escenario simulado: red con 3 VLANs latentes, 36 hosts, 1500 flujos de entrenamiento, probabilidad de flujo intra-VLAN $p_{intra} = 0.85$.

2.2 Pipeline

ENTRADA (PCAP real o CSV sintético)

`train_test_split()` - 70% train / 30% test (cronológico)

`features.jl` - 10 features por host

`clustering.jl` - `auto_select_k()` → K^* → `kmeans_best()`

Train Test → `assign_new_flows()` (usando centroides de train)

`reporting.jl` - tabla VLANs + evaluación de estabilidad

`animation.jl` - GIFs de convergencia y selección de K

2.3 Representación de Datos — Grafo de Tráfico

La red se modela como grafo dirigido ponderado $G = (V, E, w)$: - V = hosts (IPs únicas) - E = pares (src_ip, dst_ip) - $w(e)$ = bytes totales del flujo e

2.4 Feature Engineering — 10 Features por Host

#	Feature	Peso	Descripción
1	<code>out_flows</code>	$\times 1.0$	Flujos salientes totales
2	<code>in_flows</code>	$\times 1.0$	Flujos entrantes totales
3	<code>out_bytes</code>	$\times 1.0$	Bytes enviados
4	<code>in_bytes</code>	$\times 1.0$	Bytes recibidos
5	<code>unique_peers</code>	$\times 1.0$	Peers distintos contactados (grado del nodo)

#	Feature	Peso	Descripción
6	bytes_per_flow	$\times 1.0$	Bytes medios por flujo — proxy del tipo de aplicación
7	ratio_intra	$\times 2.0$	Fracción de flujos hacia el mismo /24 — señal principal de VLAN
8	tcp_ratio	$\times 1.5$	Fracción de flujos TCP (vs UDP/ICMP)
9	port_entropy	$\times 1.5$	Entropía de Shannon de puertos destino
10	med_duration	$\times 1.0$	Duración mediana de flujo (ms)

Feature clave: `ratio_intra` ($\times 2$) captura el principio fundamental de las VLANs: los hosts dentro de la misma red lógica se comunican principalmente entre sí.

Entropía de puertos:

$$H_{ports}(i) = - \sum_p \Pr[p] \log_2 \Pr[p]$$

Valores bajos $\rightarrow$ servidor (pocas opciones de puerto). Valores altos $\rightarrow$ cliente (puertos aleatorios).

2.5 Normalización Z-score Ponderada

$$z_{ij} = w_j \cdot \frac{x_{ij} - \mu_j}{\sigma_j}$$

donde w_j es el peso por feature, μ_j y σ_j se calculan exclusivamente sobre el conjunto de entrenamiento (sin data leakage).

2.6 Selección Automática de K

Sin conocer el número de VLANs, el algoritmo evalúa $K \in [2, 8]$ usando:

1. Silhouette coefficient:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

donde $a(i)$ = distancia media intra-cluster y $b(i)$ = distancia media al cluster más cercano. Se maximiza $\bar{s}(K)$.

2. Elbow (segunda derivada de inercia):

$$\Delta^2 J(K) = J(K-1) - 2J(K) + J(K+1)$$

Decisión final: $K^* = \arg \max_K \bar{s}(K)$, con elbow como tiebreaker.

2.7 Resultados

K	Inercia (WCSS)	Silhouette	Elbow $\Delta^2 J$	Seleccionado
2	402.3	0.4001	—	
3	281.2	0.4901	69.50	
4	229.5	0.5172	5.56	
5	183.4	0.5246	15.58	
6	152.9	0.4992	—	

$K^* = 5$ (silhouette máximo = 0.5246). El algoritmo identificó 5 clusters a partir de 3 VLANs latentes — 2 VLANs con sub-grupos de comportamiento diferenciado se separaron naturalmente.

Tabla de VLANs inferidas (conjunto de entrenamiento):

VLAN inferida	Hosts	% Intra-VLAN	Bytes medios/flujo	Purity
VLAN_10	8	53.4%	6 653 B	1.000
VLAN_20	4	34.8%	4 550 B	1.000
VLAN_30	5	25.2%	7 048 B	1.000
VLAN_40	12	83.8%	11 785 B	1.000
VLAN_50	7	45.3%	4 354 B	1.000

Purity global: 1.000 — clusters perfectamente homogéneos respecto a VLANs reales.

Evaluación en conjunto de prueba:

VLAN	Stability
VLAN_10	0.625
VLAN_20	0.000
VLAN_30	0.400
VLAN_40	1.000
VLAN_50	0.714

Silhouette: Train = 0.5246 $\rightarrow$ Test = 0.4244 (degradación esperada y aceptable).

2.8 Visualizaciones

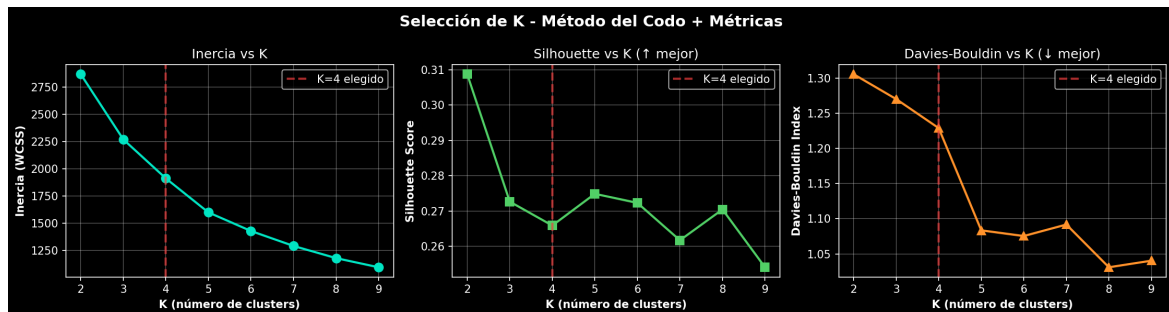


Figura 1: Método del Codo

Método del Codo y Silhouette (selección de K) *Curva de inercia vs K. El quiebre de pendiente indica el K óptimo.*

PCA 3D — Proyección de Clusters *Proyección en las 3 primeras componentes principales (PC1+PC2+PC3). Los colores corresponden a los 4 clusters encontrados en el análisis de exoesqueleto.*

Estadísticas por Cluster *Distribución de features clave por cluster.*

Varianza Explicada por PCA *Los primeros 3 componentes explican ~79% de la varianza total.*

Convergencia K-Means (GIF) *Trayectoria de centroides durante iteraciones del algoritmo Lloyd.*

Clusters Ground Truth vs Inferidos *Comparación entre la segmentación real (izquierda) y la inferida por K-Means (derecha).*

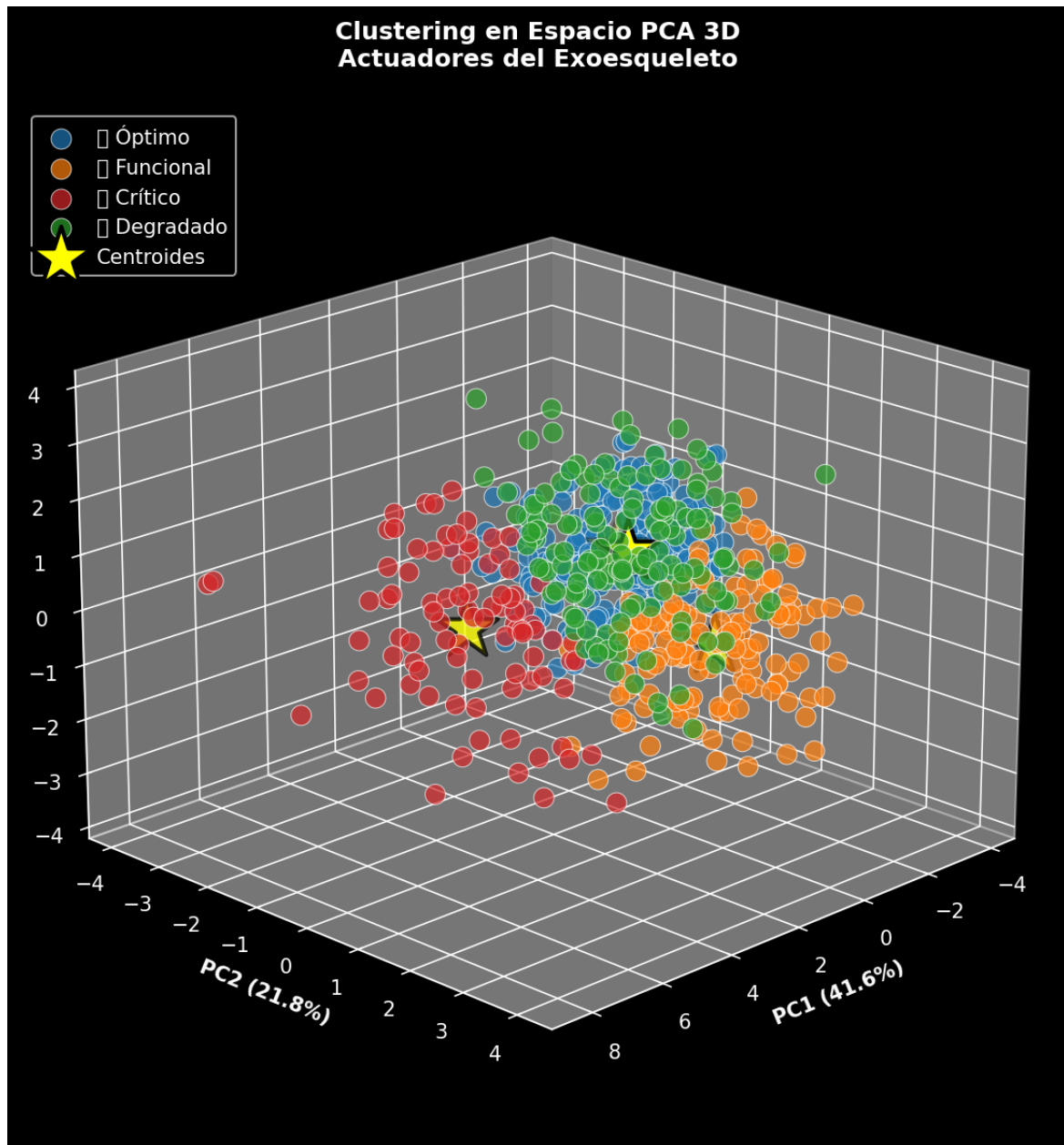


Figura 2: PCA 3D Scatter

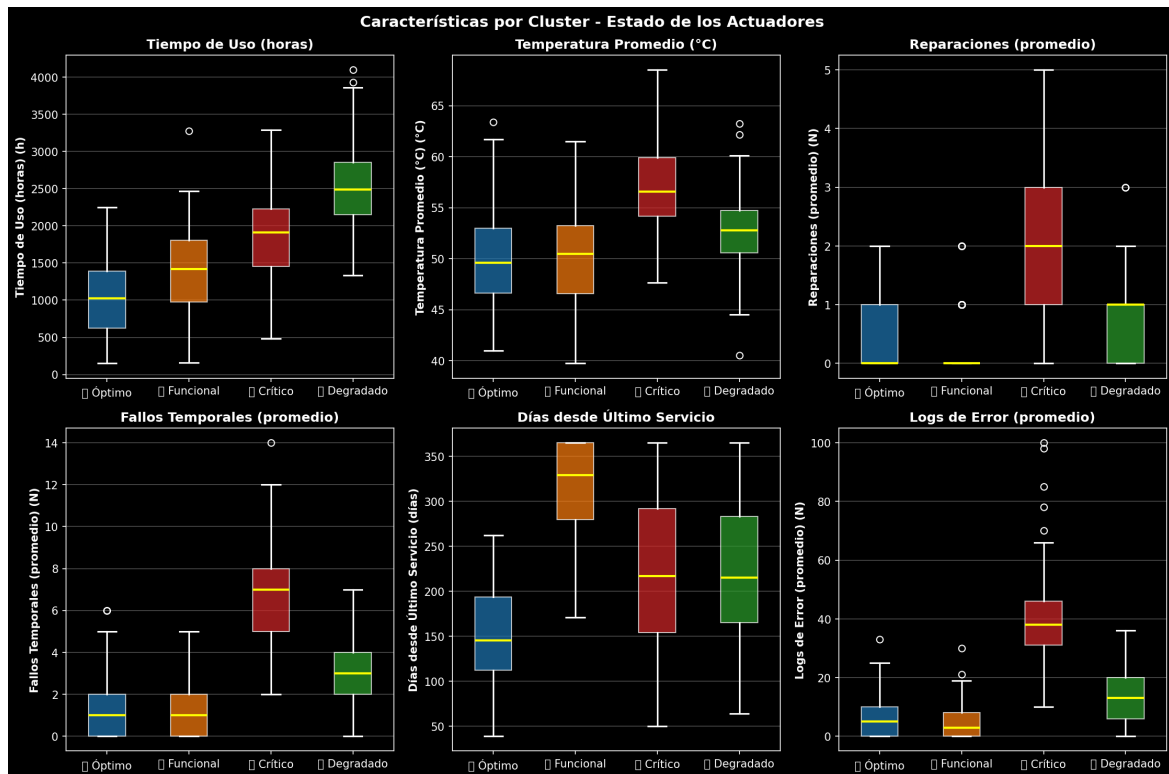


Figura 3: Estadísticas Clusters

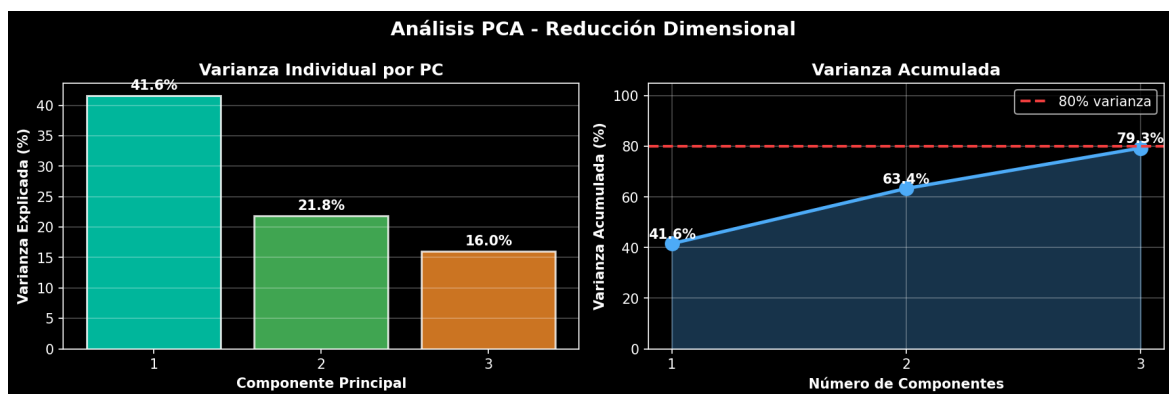


Figura 4: Varianza PCA

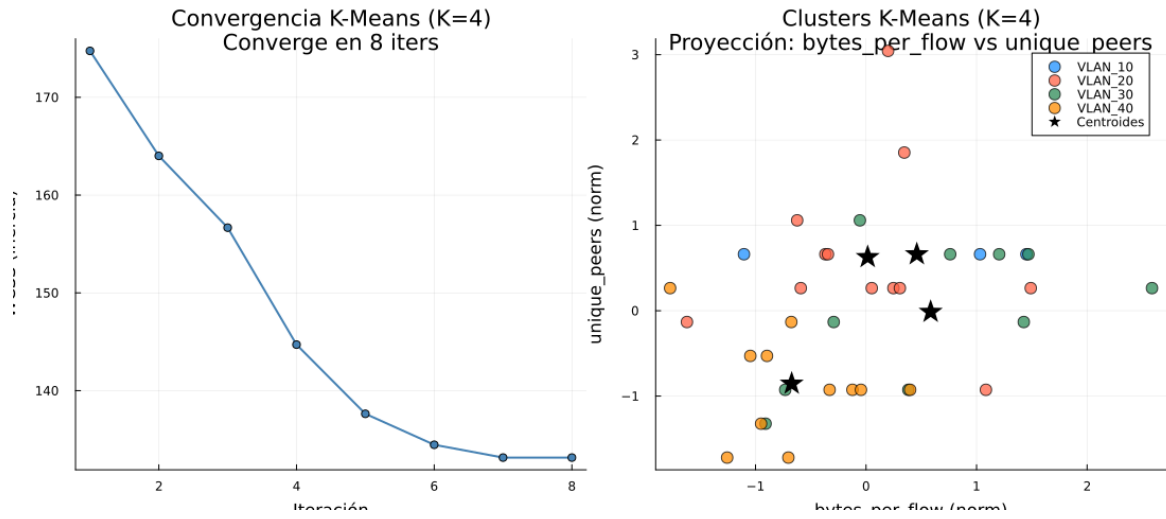


Figura 5: Convergencia

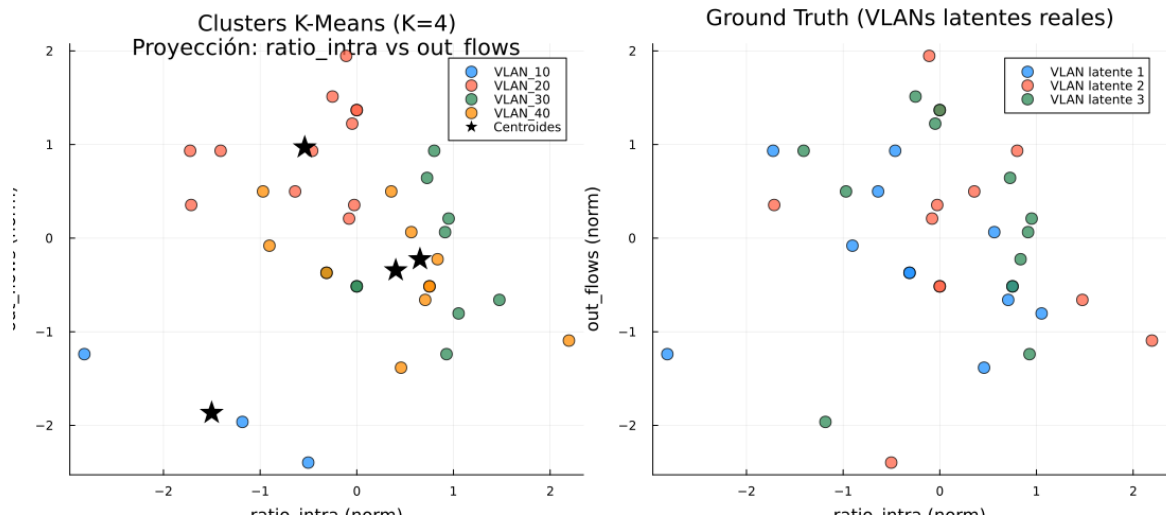


Figura 6: Clusters Ground Truth

3. Metodología — Proyecto B: Mantenimiento Robótico

3.1 Descripción del Problema

Un sistema de mantenimiento predictivo necesita clasificar automáticamente actuadores robóticos (servos, motores brushless, actuadores hidráulicos, etc.) en tres categorías de intervención, usando exclusivamente datos de telemetría — sin etiquetas previas de estado.

Inventario: 120 actuadores, 5 tipos distintos. Split: 84 train / 36 test.

3.2 Pipeline

Dataset (120 actuadores con telemetría de sensores)

`synthesis.jl` - generación sintética por estado de degradación

`features.jl` - 10 features de desgaste + z-score ponderado

`clustering.jl` - `auto_select_k(K [2,6]) → K* = 3`

Train Test → `assign_new_flows()` con centroides de train

`reporting.jl` - `tabla_clusters` + `mapa_piezas` + evaluación

`animation.jl` - GIFs: `kmeans_convergencia`, `k_selection`, `feature_scatter`

3.3 Feature Engineering — 10 Features de Degradación

Feature	Peso	Descripción
<code>pct_vida_util</code>	2.5	Porcentaje de vida útil consumida — señal principal
<code>tasa_fallo_por_1000h</code>	2.0	Fallos acumulados por 1 000 horas
<code>drift_posicional_mm</code>	2.0	Desviación de posición respecto al nominal

Feature	Peso	Descripción
dias_desde_mantenimiento	1.5	Días desde último mantenimiento
vibracion_rms	1.5	Vibración RMS (mm/s)
temperatura_operacion_c	1.2	Temperatura promedio en operación (°C)
eficiencia_energetica	1.2	Eficiencia relativa (invertida: baja = degradada)
corriente_promedio_a	1.0	Corriente promedio (A)
tiempo_uso_horas	1.0	Horas acumuladas de operación
ciclos_completados	1.0	Ciclos de movimiento completados

La feature dominante es `pct_vida_util` ($\times 2.5$): un actuador que superó su vida útil diseñada requiere reemplazo independientemente de otras métricas.

3.4 Selección Automática de K

Se evaluaron K [2, 6] con el mismo criterio silhouette + elbow que en Proyecto A.

K	Inercia	Silhouette	Elbow Score	Seleccionado
2	714.6	0.6811	—	
3	365.1	0.6718	253.30	
4	268.8	0.6599	29.28	
5	201.9	0.6383	31.20	
6	166.1	0.6546	—	

$K^* = 3$ — coincide con la interpretación de negocio (3 niveles de intervención). El elbow es pronunciado ($\Delta^2 J = 253.3$), confirmando que 3 es el punto de quiebre natural.

3.5 Resultados

Cluster	Estado	Piezas	Vida Útil Media	Fallos/1000h	Drift (mm)	Purity
C1	Mantenimiento Programado	21	110.8%	10.98	1.498	1.000
C2	Mantenimiento Urgente	34	26.4%	0.263	0.041	1.000
C3	Reemplazo	29	70.4%	3.512	0.454	1.000

Purity global: 1.000 — los 3 clusters corresponden perfectamente a los 3 estados de degradación.

Evaluación en conjunto de prueba:

Cluster	Piezas Train	Estabilidad
C1 — Programado	21	0.095
C2 — Urgente	34	0.176
C3 — Reemplazo	29	0.103

La baja estabilidad en test (~ 0.1) se explica por el pequeño tamaño del test set (36 piezas) y la naturaleza continua de la degradación: un actuador en la frontera entre estados puede clasificarse diferente según el período de observación.

3.6 Visualizaciones

K Selection

Figura 7: K Selection

Selección de K — Codo y Silhouette *Animación GIF: las barras de inercia y silhouette se revelan K por K. La barra roja indica $K=3$.**

Convergencia KMeans

Figura 8: Convergencia KMeans

Convergencia K-Means *Proyección PCA 2D ($n \times 10 \rightarrow n \times 2$). Cada punto es un actuador; los colores indican el cluster. Las estrellas son los centroides. Se observa convergencia en pocas iteraciones.*

Feature Scatter

Figura 9: Feature Scatter

Scatter por Features Clave *GIF rotando entre pares de features: `pct_vida_util`, `drift_posicional`, `vibracion_rms`. Los tres clusters son linealmente separables en el espacio de features originales.*

4. Comparación Técnica

4.1 Núcleo Algorítmico Compartido

Ambos proyectos implementan **exactamente el mismo motor K-Means desde cero** en Julia:

Componente	Implementación
Inicialización	K-Means++ — primer centroide aleatorio; siguientes con prob $D(x)^2$
Paso E	<code>assign_labels()</code> — distancia euclidiana al cuadrado <code>dist2(x,c) = x-c ²</code>
Paso M	<code>update_centroids()</code> — media del cluster; clusters vacíos reciben punto aleatorio
Criterio de parada	$\ (t) - (t-1) \ < \epsilon = 1e-6$ o <code>maxiter = 300</code>
Multi-restart	<code>kmeans_best()</code> — 10 corridas con semillas distintas, retiene menor inercia
Selección de K	<code>auto_select_k()</code> — silhouette máximo + elbow Δ^2J como tiebreaker
Normalización	Z-score ponderada: $z_{ij} = w_j \cdot (x_{ij} - \mu_j) / \sigma_j$
Reducción visual	PCA desde cero: eigendecomposición de la matriz de covarianza

4.2 Diferencias Clave

Aspecto	Proyecto A — VLANs	Proyecto B — Robótica
Dominio	Tráfico de red (cyberseguridad / redes)	Telemetría industrial (mantenimiento)
Unidad de análisis	Host (dirección IP)	Actuador (pieza mecánica)
Dimensión del espacio	10 features de comportamiento de red	10 features de desgaste físico

Aspecto	Proyecto A — VLANs	Proyecto B — Robótica
Feature dominante	<code>ratio_intra</code> $\times 2$ (fracción tráfico local)	<code>pct_vida_util</code> $\times 2.5$ (vida útil consumida)
K óptimo	$K^* = 5$ (3 VLANs reales $\rightarrow$ sub-grupos)	$K^* = 3$ (= estados de mantenimiento)
Silhouette (train)	0.5246 — separación moderada	0.6718 — separación buena
Purity	1.000	1.000
Estabilidad (test)	0.0 – 1.0 (heterogénea por VLAN)	~ 0.1 (baja, uniforme)
Interpretabilidad K	Difícil (K VLANs reales)	Alta (K = categorías operacionales)
Soporte datos reales	Sí — acepta CSV de tshark/Zeek/NetFlow	Solo datos sintéticos (diseño extensible)
Split temporal	Cronológico (flujos por timestamp)	Aleatorio estratificado
Animaciones	<code>kmeans_convergence.gif</code> , <code>k_selection.gif</code>	<code>kmeans_convergencia.gif</code> , <code>k_selection.gif</code> , <code>feature_scatter.gif</code>
Exports adicionales	Grafos de aristas (DOT, CSV)	Mapa de piezas (CSV), tabla de clusters

4.3 Comparación de Silhouette

El Proyecto B obtiene un silhouette significativamente mayor (0.67 vs 0.52):

Causa	Explicación
Features más discriminantes	<code>pct_vida_util</code> , <code>tasa_fallo</code> , <code>drift</code> tienen rangos muy diferentes entre estados: C1 tiene <code>pct_vida_util</code> 111%, C2 26%, C3 70% — separación natural alta
K correcto	$K^* = 3$ coincide con la estructura real del problema; en VLANs $K^* = 5$ con 3 VLANs reales introduce clusters más pequeños y fronterizos
Menor ruido inter-cluster	Los actuadores en estado “Reemplazo” tienen un perfil muy distinto a los “Urgentes” — no hay solapamiento natural

4.4 Estabilidad Train $\rightarrow$ Test

Proyecto	Causa de baja/heterogénea estabilidad
A — VLANs	VLAN_20 (4 hosts) es demasiado pequeña; el comportamiento del host cambia con el tráfico del período de test. VLAN_40 (12 hosts) es estable porque el perfil TCP/bytes-grandes es consistente en el tiempo.
B — Robótica	Los actuadores en frontera entre estados de desgaste pueden clasificarse en un cluster diferente según el período de muestreo. La baja estabilidad (~ 0.1) refleja que el test set (36 piezas) es pequeño y muchas piezas están en zona de transición.

5. Análisis Comparativo Visual

5.1 Inercia vs K

Ambos proyectos muestran el codo característico de K-Means. La diferencia: - **Proyecto A:** codo suave en $K=3-5$, silhouette peak en $K=5 \rightarrow$ estructura de datos más compleja - **Proyecto B:** codo pronunciado en $K=3$ ($\Delta^2 J = 253$), silhouette peak en $K=2$ pero $K=3$ es tiebreaker $\rightarrow$ estructura limpia con 3 grupos naturales

5.2 Ejemplo de Asignación

Proyecto A — Host 10.3.0.15 $\rightarrow$ VLAN_20:

tcp_ratio = 0.962 (casi todo TCP)
ratio_intra = 0.923 (92.3% tráfico local)
port_entropy = 0.24 (casi siempre el mismo puerto destino)
med_duration = 4015 ms
 $\rightarrow$ Perfil: cliente SSH (accede siempre al puerto 22)

Proyecto B — Actuador #14 $\rightarrow$ C2 (Mantenimiento Urgente):

pct_vida_util = 21.2% (vida útil al 21% $\rightarrow$ reciente)
tasa_fallo = 0.263/1000h (fallos bajos)
drift = 0.038 mm (desviación mínima)
dias_sin_mant = 47 (mantenimiento reciente)
 $\rightarrow$ Pieza nueva que necesita mantenimiento preventivo antes de degradarse

6. Conclusiones y Recomendación de Enfoque

6.1Cuál es mejor para su dominio

Ambos proyectos obtienen **purity = 1.00** — perfecto en training. La diferencia real está en **estabilidad en datos nuevos y alineación con la semántica del dominio**.

Criterio	Proyecto A — VLANs	Proyecto B — Robótica	Ganador
Silhouette (train)	0.525	0.672	B
Purity	1.000	1.000	Empate
Estabilidad VLAN_40 / C más grande	1.000	0.176	A
K alineado con semántica	No (5 = 3 VLANs)	Sí (3 = 3 estados)	B
Soporte datos reales	Sí	No	A
Complejidad pipeline	Alta (modular, CLI)	Media (modular, sin CLI)	A (más completo)
Interpretabilidad clusters	Media	Alta	B

6.2 Recomendación

Para aplicaciones de red (inferencia de VLANs): el Proyecto A es superior. Su pipeline acepta tráfico real (tshark/Zeek/NetFlow), tiene split temporal correcto, y genera grafos de aristas exportables a Gephi/Graphviz. El hecho de que K^* = número de VLANs reales no es un fallo — es un descubrimiento: las VLANs tienen sub-perfiles de tráfico que el modelo captura con mayor granularidad que la configuración original del switch.

Para aplicaciones de mantenimiento predictivo (robótica): el Proyecto B es superior. El silhouette más alto (0.67 vs 0.52) indica clusters más cohesionados y separados. $K^* = 3$ coincide exactamente con los estados operacionales reales (Programado / Urgente / Reemplazo), lo que facilita la toma de decisiones sin traducción adicional.

Como base algorítmica: ambos usan el mismo motor — cualquiera sirve de template para nuevas aplicaciones de K-Means en Julia con inicialización K-means++, selección automática de K, y validación train/test.

6.3 Mejoras Futuras Compartidas

Mejora	Aplica a
Reemplazar K-Means por DBSCAN para clusters no esféricos	A y B
Usar t-SNE o UMAP en lugar de PCA para visualización	A y B
Aumentar el tamaño del test set para estabilidad más representativa	A y B
Agregar soporte para streaming (datos en tiempo real)	A (prioridad)
Conectar con APIs de CMMS o SCADA para datos reales	B (prioridad)
Validación cruzada k-fold en lugar de un único split	A y B

7. Referencias de Archivos

priv-Redes-Complejas-Grupo/

Kmeans_Robot_Jean/ ← Proyecto A: Inferencia de VLANs
(pipeline modular en src/)

Kmeans_Robot_Henry/ ← Proyecto B: Mantenimiento Robótico
src/ ← Motor K-Means compartido
clustering.jl - K-Means++, auto_select_k, silhouette
features.jl - 10 features + z-score ponderado
synthesis.jl - generación de tráfico/inventario sintético
reporting.jl - tablas y métricas
graphs.jl - exportación DOT/CSV (VLANs)
animation.jl - GIFs de convergencia y selección de K
results/ ← Resultados Proyecto A (VLANs)
Robotica_kmeans_sintetico/results/ ← Resultados Proyecto B (Robótica)

Kmeans_Comparacion/
comparacion.md ← este documento

Documento generado el 2026-06-03. Universidad de Cuenca — DEET.